

UNIVERSIDAD DE COSTA RICA

ESCUELA DE INGENIERÍA ELÉCTRICA

IE0117: PROGRAMACIÓN BAJO PLATAFORMAS ABIERTAS

Reporte

Laboratorio #5

Prof. Carolina Trejos Quirós

Estudiante: Oriana Mora López, Carne B75079

1 de julio de 2026

Índice

1. Introducción	2
2. Implementación y Resultados	2
2.1. Arreglo Dinámico	2
2.2. Lista Enlazada Simple	2
2.3. Lista Doblemente Enlazada	3
2.4. Stack	3
2.5. Verificación de Memory Leaks	4
3. Discusión de Resultados y Conclusiones	6

al inicio, y 30 al final. Luego se insertó el valor 25 en la posición 1, se eliminó el nodo con valor 20, y se realizó una búsqueda del valor 30.

```

PS C:\Users\HP\Lab-5-_-Progra-I-2026> .\main.exe
Arreglo: 10 20 30 40
Lista: 20 -> 10 -> 30 -> NULL
Lista: 20 -> 25 -> 10 -> 30 -> NULL
Lista: 25 -> 10 -> 30 -> NULL
Elemento encontrado: 30

```

Figura 2: Resultado 2 [?].

Como se observa en la figura anterior, cada operación modificó correctamente la estructura de la lista: la inserción en la posición 1 colocó el valor 25 entre los nodos 20 y 10, la eliminación del nodo 20 desconectó correctamente el nodo sin afectar el resto de la lista, y la búsqueda encontró exitosamente el valor 30.

2.3. Lista Doblemente Enlazada

Se implementó una lista doblemente enlazada, la cual extiende la lista simple agregando un puntero al nodo anterior en cada nodo. Esto permite recorrer la lista en ambas direcciones. Se implementaron las mismas operaciones que en la lista simple, agregando la función de impresión en orden inverso. Se realizaron las mismas pruebas que en la lista enlazada simple, verificando adicionalmente la impresión hacia atrás.

```

PS C:\Users\HP\Lab-5-_-Progra-I-2026> .\main.exe
Arreglo: 10 20 30 40
Lista: 20 -> 10 -> 30 -> NULL
Lista: 20 -> 25 -> 10 -> 30 -> NULL
Lista: 25 -> 10 -> 30 -> NULL
Elemento encontrado: 30
Lista (adelante): 20 <-> 10 <-> 30 <-> NULL
Lista (atras): 30 <-> 10 <-> 20 <-> NULL
Lista (adelante): 20 <-> 25 <-> 10 <-> 30 <-> NULL
Lista (adelante): 25 <-> 10 <-> 30 <-> NULL
Elemento encontrado (doble): 30

```

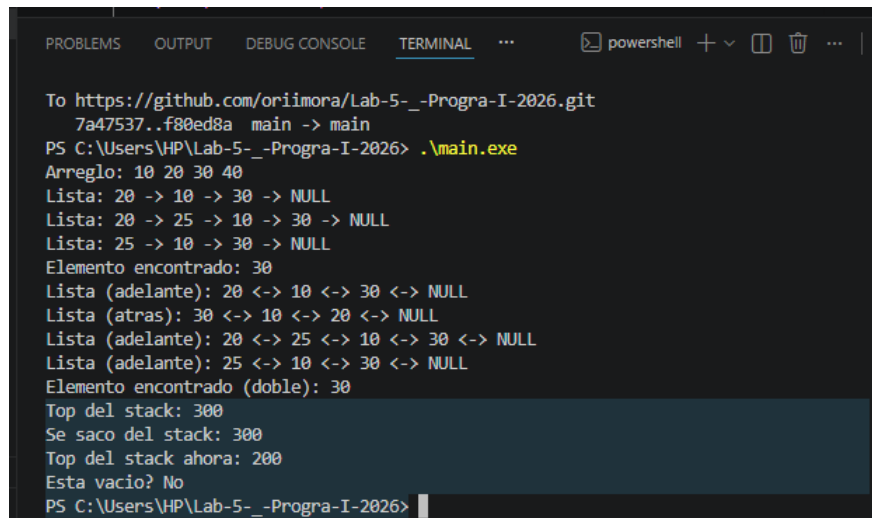
Figura 3: Resultado 3 [?].

Como se observa en la figura anterior, la impresión hacia atrás mostró los elementos en orden inverso al de la impresión hacia adelante (30, 10, 20), confirmando que los punteros **prev** se actualizaron correctamente en cada operación de inserción y eliminación.

2.4. Stack

Se implementó una pila (stack) utilizando una lista enlazada internamente, donde el puntero **top** señala siempre al último elemento agregado. Se implementaron las operaciones **push**, **pop**, **peek** e **Empty**.

Para probar su funcionamiento, se agregaron los valores 100, 200 y 300 mediante **push**. Posteriormente se consultó el tope de la pila, se extrajo un elemento con **pop**, y se volvió a consultar el nuevo tope.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL ... powershell + - [ ] [X] ... |
To https://github.com/oriimora/Lab-5-_Progra-I-2026.git
7a47537..f80ed8a main -> main
PS C:\Users\HP\Lab-5-_Progra-I-2026> .\main.exe
Arreglo: 10 20 30 40
Lista: 20 -> 10 -> 30 -> NULL
Lista: 20 -> 25 -> 10 -> 30 -> NULL
Lista: 25 -> 10 -> 30 -> NULL
Elemento encontrado: 30
Lista (adelante): 20 <-> 10 <-> 30 <-> NULL
Lista (atras): 30 <-> 10 <-> 20 <-> NULL
Lista (adelante): 20 <-> 25 <-> 10 <-> 30 <-> NULL
Lista (adelante): 25 <-> 10 <-> 30 <-> NULL
Elemento encontrado (doble): 30
Top del stack: 300
Se saco del stack: 300
Top del stack ahora: 200
Esta vacio? No
PS C:\Users\HP\Lab-5-_Progra-I-2026>
```

Figura 4: Resultado 4 [?].

Como se observa en la Figura anterior, el comportamiento del stack correspondió al esperado bajo la política LIFO (Last In, First Out): el valor 300, al ser el último insertado, fue tanto el tope inicial como el primer elemento extraído mediante `texttttop`, quedando el valor 200 como nuevo tope.

2.5. Verificación de Memory Leaks

Para lograr el correcto manejo de la memoria dinámica, se utilizó la herramienta Valgrind sobre el ejecutable compilado en un entorno Linux (Ubuntu, mediante máquina virtual), ejecutando el siguiente comando:

```

Ori_VM [Corriendo] - Oracle VirtualBox
mié 1 de jul 12:01 A.M.

crimora@Ori_VM:~/Lab-5_Progra-1-2023$ valgrind --leak-check=full ./main
==6133== Memcheck, a memory error detector
==6133== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==6133== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==6133== Command: ./main
==6133==
Arreglo: 10 20 30 40
Lista: 20 -> 10 -> 30 -> NULL
Lista: 20 -> 25 -> 10 -> 30 -> NULL
Lista: 25 -> 10 -> 30 -> NULL
Elemento encontrado: 30
Lista (adelante): 20 <-> 10 <-> 30 <-> NULL
Lista (atras): 30 <-> 10 <-> 20 <-> NULL
Lista (adelante): 20 <-> 25 <-> 10 <-> 30 <-> NULL
Lista (adelante): 25 <-> 10 <-> 30 <-> NULL
Elemento encontrado (doble): 30
Top del stack: 300
Se saco del stack: 300
Top del stack ahora: 200
Esta vacio? No
==6133==
==6133== HEAP SUMMARY:
==6133==    in use at exit: 0 bytes in 0 blocks
==6133==   total heap usage: 16 allocs, 16 frees, 1,292 bytes allocated
==6133==
==6133== All heap blocks were freed -- no leaks are possible
==6133==
==6133== For lists of detected and suppressed errors, rerun with: -s
==6133== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
crimora@Ori_VM:~/Lab-5_Progra-1-2023$ valgrind --leak-check=full ./main > valgrind_report.txt 2>1
crimora@Ori_VM:~/Lab-5_Progra-1-2023$ cat valgrind_report.txt
Orden «valgrind» no encontrada. Quizá quiso decir:
  * la orden «valgrind» del paquete snap «valgrind (3.26.0)»
  * la orden «valgrind» del paquete deb «valgrind (1:3.22.0-0ubuntu2)»
Consulte «snap info <nombre del snap>» para ver más versiones.
crimora@Ori_VM:~/Lab-5_Progra-1-2023$ valgrind --leak-check=full ./main > valgrind_report.txt 2>1
crimora@Ori_VM:~/Lab-5_Progra-1-2023$ cat valgrind_report.txt
==6568== Memcheck, a memory error detector
==6568== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==6568== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==6568== Command: ./main
==6568==
Arreglo: 10 20 30 40
Lista: 20 -> 10 -> 30 -> NULL
Lista: 20 -> 25 -> 10 -> 30 -> NULL
Lista: 25 -> 10 -> 30 -> NULL
Elemento encontrado: 30
Lista (adelante): 20 <-> 10 <-> 30 <-> NULL
Lista (atras): 30 <-> 10 <-> 20 <-> NULL
Lista (adelante): 20 <-> 25 <-> 10 <-> 30 <-> NULL
Lista (adelante): 25 <-> 10 <-> 30 <-> NULL
Elemento encontrado (doble): 30
Top del stack: 300
Se saco del stack: 300
Top del stack ahora: 200
Esta vacio? No
==6568==
==6568== HEAP SUMMARY:
==6568==    in use at exit: 0 bytes in 0 blocks
==6568==   total heap usage: 16 allocs, 16 frees, 4,364 bytes allocated
==6568==
==6568== All heap blocks were freed -- no leaks are possible
==6568==
==6568== For lists of detected and suppressed errors, rerun with: -s
==6568== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
crimora@Ori_VM:~/Lab-5_Progra-1-2023$ git add valgrind_report.txt
crimora@Ori_VM:~/Lab-5_Progra-1-2023$ git commit -m "Agregar reporte de valgrind sin memory leaks"
[develop 8bd7cf9] Agregar reporte de valgrind sin memory leaks
 1 file changed, 28 insertions(+)
 create mode 100644 valgrind_report.txt
crimora@Ori_VM:~/Lab-5_Progra-1-2023$ git push

```

Figura 5: Resultado 5 [?].

```

Ori_VM [Corriendo] - Oracle VirtualBox
mié 1 de jul 12:01 A.M.

crimora@Ori_VM:~/Lab-5_Progra-1-2023$ valgrind --leak-check=full ./main > valgrind_report.txt 2>1
crimora@Ori_VM:~/Lab-5_Progra-1-2023$ cat valgrind_report.txt
==6568== Memcheck, a memory error detector
==6568== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==6568== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==6568== Command: ./main
==6568==
Arreglo: 10 20 30 40
Lista: 20 -> 10 -> 30 -> NULL
Lista: 20 -> 25 -> 10 -> 30 -> NULL
Lista: 25 -> 10 -> 30 -> NULL
Elemento encontrado: 30
Lista (adelante): 20 <-> 10 <-> 30 <-> NULL
Lista (atras): 30 <-> 10 <-> 20 <-> NULL
Lista (adelante): 20 <-> 25 <-> 10 <-> 30 <-> NULL
Lista (adelante): 25 <-> 10 <-> 30 <-> NULL
Elemento encontrado (doble): 30
Top del stack: 300
Se saco del stack: 300
Top del stack ahora: 200
Esta vacio? No
==6568==
==6568== HEAP SUMMARY:
==6568==    in use at exit: 0 bytes in 0 blocks
==6568==   total heap usage: 16 allocs, 16 frees, 4,364 bytes allocated
==6568==
==6568== All heap blocks were freed -- no leaks are possible
==6568==
==6568== For lists of detected and suppressed errors, rerun with: -s
==6568== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
crimora@Ori_VM:~/Lab-5_Progra-1-2023$ git add valgrind_report.txt
crimora@Ori_VM:~/Lab-5_Progra-1-2023$ git commit -m "Agregar reporte de valgrind sin memory leaks"
[develop 8bd7cf9] Agregar reporte de valgrind sin memory leaks
 1 file changed, 28 insertions(+)
 create mode 100644 valgrind_report.txt
crimora@Ori_VM:~/Lab-5_Progra-1-2023$ git push

```

Figura 6: Resultado 6 [?].

Como se observa en la figura anterior, el resumen de memoria (*HEAP SUMMARY*) reporta 16 reservas de memoria (*allocs*) y 16 liberaciones (*frees*), con el mensaje *All heap blocks were*

freed – no leaks are possible" y un total de 0 errores. Esto confirma que toda la memoria reservada dinámicamente mediante `malloc` y `realloc` en las cuatro estructuras de datos fue correctamente liberada mediante las funciones `freeArray`, `freeList`, `freeListD` y `freeStack`.

3. Discusión de Resultados y Conclusiones

Discusión de resultados

A partir de la implementación y prueba de las cuatro estructuras de datos, se pudo comprobar el correcto funcionamiento de cada una de las operaciones solicitadas. El arreglo dinámico demostró un manejo adecuado del redimensionamiento automático mediante `realloc`, permitiendo agregar elementos más allá de su capacidad inicial sin pérdida de información. Por su parte, tanto la lista enlazada simple como la doblemente enlazada mostraron un comportamiento consistente en las operaciones de inserción, eliminación y búsqueda, siendo la principal diferencia entre ambas la capacidad de recorrido bidireccional que ofrece el puntero `prev` en la lista doble, verificada mediante la impresión en orden inverso. En cuanto al stack, su comportamiento LIFO se validó correctamente: el último elemento insertado (300) fue el primero en salir mediante `pop`, comportamiento esperado para este tipo de estructura y consistente con su implementación interna basada en una lista enlazada donde `top` siempre referencia al nodo más recientemente agregado.

Un evento importante del desarrollo fue la gestión de memoria dinámica. Inicialmente, se detectó que, si bien existían funciones de liberación de memoria (`freeArray`, `freeList`), estas no estaban siendo invocadas desde `main.c`, lo cual habría generado fugas de memoria al finalizar la ejecución del programa. Tras corregir este punto e incluir las llamadas correspondientes a las funciones de liberación para las cuatro estructuras, la verificación con Valgrind confirmó que la totalidad de la memoria reservada (16 bloques) fue correctamente liberada, sin errores ni fugas detectadas.

Finalmente, en cuanto al manejo del repositorio, la organización del proyecto mediante ramas independientes por estructura de datos, siguiendo el flujo de trabajo GitFlow, permitió mantener un historial de commits ordenado e incremental para cada funcionalidad desarrollada, facilitando la trazabilidad de los cambios realizados a lo largo del laboratorio.

Conclusiones

El desarrollo de este laboratorio permitió reforzar los conceptos de memoria dinámica en el lenguaje C mediante la implementación práctica de cuatro estructuras de datos fundamentales: arreglo dinámico, lista enlazada simple, lista doblemente enlazada y stack. Cada una de estas estructuras cumplió correctamente con las operaciones solicitadas, lo cual se verificó tanto mediante pruebas funcionales en `main.c` como a través del análisis de memoria con Valgrind.

Además, el laboratorio permitió afianzar el uso de Git como herramienta de control de versiones, particularmente el flujo de trabajo GitFlow, mediante la creación de ramas independientes por cada estructura de datos y la integración progresiva del código hacia las ramas `develop` y `main`. Este proceso evidenció la importancia de mantener un historial de commits claro y conforme al avance, así como de subir el trabajo regularmente al repositorio remoto para evitar la pérdida de avances.

Finalmente, la verificación con Valgrind confirmó la ausencia total de fugas de memoria (0 errores, 16 reservas y 16 liberaciones), lo que demuestra un manejo adecuado y responsable de la memoria dinámica en las cuatro estructuras implementadas.

Recomendaciones

- Realizar commits de Git de forma frecuente durante el desarrollo, incluso para cambios pequeños, con el fin de mantener un historial detallado del progreso y facilitar la recuperación de versiones anteriores en caso de errores.
- Verificar el manejo de memoria con Valgrind durante cada etapa o antes de hacer commits nuevos, y no únicamente al finalizar la implementación, para detectar y corregir fugas de memoria de forma oportuna.
- Documentar el propósito de cada función mediante comentarios en el código fuente, especialmente en operaciones que involucran manipulación de punteros, para facilitar su comprensión y mantenimiento futuro.